1) List three data sources Power BI can connect to

- Excel workbook (`.xlsx`)
- SQL Server database
- Web/URL (HTML tables, JSON APIs)

There are many more: CSV, SharePoint/OneDrive, Oracle, MySQL, Parquet, etc.

---

2) What is the first step to import data into Power BI Desktop?

Home ➜ Get data ➜ choose your source (e.g., **Text/CSV** for a `.csv` file), then pick the file and proceed.

---

3) How do you refresh imported data in Power BI?

- **Power BI Desktop:** Home ➜ **Refresh** (or right-click a table ➜ **Refresh data**).
- **Power BI Service:** Dataset ➜ **Refresh now** or set **Scheduled refresh** (requires a gateway for on-prem/local files).

---

4) What file formats can Power BI import directly? (Name two.)

- **CSV (.csv)**
- **Excel (.xlsx)**
  (Also: **Text (.txt)**, **JSON (.json)**, **XML (.xml)**, **Parquet**, etc.)

---

5) What does the “Navigator” window show after selecting a data source?

It lists the available objects you can load (for files: tables/sheets or the file preview; for databases: tables/views). It also shows a **preview** pane and lets you **check** one or more items to **Load** or **Transform Data**.

---

6) Import `Sales_Data.csv` and load only the “Product” and “Price” columns

Click path (Desktop)

1. Home ➜ **Get data** ➜ **Text/CSV** ➜ select `Sales_Data.csv`.
2. In the preview dialog, click **Transform Data** (not Load).
3. In Power Query:
   - Select columns **Product** and **Price** (hold **Ctrl** to multi-select).
   - Right-click a selected header ➜ **Remove Other Columns** (now only those two remain).
4. Home ➜ **Close & Apply**.

Equivalent M code (for reference)

```
let
    Source = Csv.Document(
        File.Contents("C:\Data\Sales_Data.csv"),
        [Delimiter=",", Columns=5, Encoding=65001, QuoteStyle=QuoteStyle.Csv]
    ),
    #"Promoted Headers" = Table.PromoteHeaders(Source,
[PromoteAllScalars=true]),
    #"Typed" = Table.TransformColumnTypes(#"Promoted Headers", {
        {"OrderID", Int64.Type}, {"Product", type text}, {"Quantity",
Int64.Type},
        {"Price", type number}, {"OrderDate", type text}
    }),
    #"Keep Product & Price" = Table.SelectColumns(#"Typed",
{"Product","Price"})
in
    #"Keep Product & Price"
```

---

7) How would you change `OrderDate` to a date format during import?

Click path

1. In Power Query, click the **OrderDate** column.

2. On the top ribbon, use the **Data type** dropdown (ABC/123 icon) ➜ choose **Date**.
    - If dates are ambiguous (e.g., `1/5/2023` could be 1 May or Jan 5), use **Transform ➜ Data type ➜ Using Locale…**
        - Type: **Date**
        - Locale:
            - **en-GB** (DMY) for day/month/year
            - **en-US** (MDY) for month/day/year
3. Apply.

M with locale (safer for ambiguous formats)

```
#"OrderDate as Date" = Table.TransformColumnTypes(#"Typed", {{"OrderDate",
type date}}, "en-GB")
```

(Use `"en-US"` if your CSV is MDY.)

---

8) What is the difference between Load and Transform Data in the import dialog?

- **Load:** Pushes the data **straight into the model** immediately (no changes).
- **Transform Data:** Opens **Power Query** so you can **clean/shape** (filter, change types, split, merge, etc.) **before** loading.

---

9) Why might you see an error when connecting to a SQL database? (Name one reason.)

Example reasons:

- **Firewall/Network** blocks the server/port.
- **Wrong server/database name** or **invalid credentials**.
- **SQL Server Browser/TCP-IP** disabled on the server.
  (Any one of these is acceptable.)

---

10) How do you replace a data source after importing it?

Two reliable ways:

1. **Power Query:** Home ➜ **Transform data** ➜ in **Query Settings**, click the **gear** icon on the **Source** step ➜ change file path/server ➜ OK.
2. **Data Source Settings:** Home ➜ **Transform data** ➜ **Data source settings** ➜ select the source ➜ **Change Source…** ➜ update path/server/database ➜ OK.

---

11) Write the M-code to import only rows where `Quantity > 1`

```
let
    Source = Csv.Document(
        File.Contents("C:\Data\Sales_Data.csv"),
        [Delimiter=",", Columns=5, Encoding=65001, QuoteStyle=QuoteStyle.Csv]
    ),
    #"Promoted Headers" = Table.PromoteHeaders(Source,
[PromoteAllScalars=true]),
    #"Changed Types" = Table.TransformColumnTypes(#"Promoted Headers", {
        {"OrderID", Int64.Type}, {"Product", type text}, {"Quantity",
Int64.Type},
        {"Price", type number}, {"OrderDate", type date}
    }),
    #"Filtered Rows" = Table.SelectRows(#"Changed Types", each [Quantity] >
1)
in
    #"Filtered Rows"
```

---

12) How would you change the data source if `Sales_Data.csv` changed?

- If **same path/filename** (you replaced the file): just **Refresh**.
- If **path or name changed:** open Power Query, **Source** step (gear) ➜ point to the new file.
- **Best practice:** create a **parameter** for the file path (Manage Parameters), then in M:
- `Source = Csv.Document(File.Contents(FilePathParameter), [Delimiter=",", Columns=5, Encoding=65001, QuoteStyle=QuoteStyle.Csv])`

  Now you can change the file once by updating the parameter value.

---

13) Troubleshoot: CSV import fails due to a “mixed data type” error — how to fix?

Cause: A column expected to be numeric (e.g., `Quantity` or `Price`) includes text in some rows (like `N/A`, blanks, extra spaces, or `-`).
Fix (Power Query):

1. Set the problematic column to **Text** first (so it won’t error).
2. **Clean** it: Trim spaces, replace `N/A`, empty strings, or stray characters.
3. Convert to **Number** using a **safe** conversion (try/otherwise), or use **Using Locale** if needed.

Robust M pattern:

```
let
    Source = Csv.Document(File.Contents("C:\Data\Sales_Data.csv"),
[Delimiter=",", Columns=5, Encoding=65001, QuoteStyle=QuoteStyle.Csv]),
    #"Promoted Headers" = Table.PromoteHeaders(Source,
[PromoteAllScalars=true]),
    // 1) Treat as Text so it never errors on read:
    #"As Text" = Table.TransformColumnTypes(#"Promoted Headers",
{{"Quantity", type text}, {"Price", type text}}),
    // 2) Clean the strings (remove spaces, swap commas->dots if needed,
remove non-numeric chars):
    #"Cleaned" = Table.TransformColumns(#"As Text", {
        {"Quantity", each Text.Select(Text.Trim(_), {"0".."9","-",".”}), type
text},
        {"Price",    each Text.Select(Text.Trim(_), {"0".."9","-",".”}), type
text}
    }),
    // 3) Safe cast to numbers (bad values become null instead of throwing
errors):
    #"To Number (safe)" = Table.TransformColumns(#"Cleaned", {
        {"Quantity", each try Number.From(_) otherwise null, Int64.Type},
        {"Price",    each try Number.From(_) otherwise null, type number}
    })
in
    #"To Number (safe)"
```

(You can then filter out nulls if necessary.)

---

14) Connect to a live SQL database with parameters (e.g., filter by year)

“Live” typically means **DirectQuery** so queries run against SQL at view time.

Set up (Desktop)

1. **Manage Parameters ➜ New Parameter**:
    - Name: `YearParam` | Type: **Whole Number** | Current Value: e.g., **2023**.

2. **Get Data ➜ SQL Server**
    - **Server**: `yourServerName`
    - **Database**: `yourDatabaseName`
    - **Data Connectivity mode**: **DirectQuery** (for live).
    - **OK**.
3. In the Queries pane, select the `Sales` table.
    - Power Query: **Filter** `OrderDate` by **Year equals** `YearParam` (use the filter UI or code below).
    - Confirm **query folding** (right-click step ➜ **View Native Query** is enabled).

Fold-friendly approach (no custom SQL; works in DirectQuery)

```
let
    Source = Sql.Database("yourServerName", "yourDatabaseName"),
    Sales  = Source{[Schema="dbo", Item="Sales"]}[Data],
    #"Filtered by Year" = Table.SelectRows(Sales, each Date.Year([OrderDate])
= YearParam)
in
    #"Filtered by Year"
```

Native SQL with a parameter (also fine)

```
let
    Source = Sql.Database("yourServerName", "yourDatabaseName"),
    Sql    = "
        SELECT OrderID, Product, Quantity, Price, OrderDate
        FROM dbo.Sales
        WHERE YEAR(OrderDate) = @yr;",
    Result = Value.NativeQuery(Source, Sql, [yr = YearParam])
in
    Result
```

(Publish to the Service; you can expose `YearParam` via a parameter and let report users change it using a What-If parameter or field parameters, or manage it before publishing.)

---

15) How would you automate data imports using Power BI + Power Automate?

Goal: Trigger a dataset refresh automatically (on a schedule or when a file changes).

Steps (Power BI Service + Power Automate):

1. In Power BI Desktop: **File ➜ Publish** your report to a workspace.
2. In **Power Automate**: **Create ➜ Automated cloud flow**.
    - **Trigger** options:
        - **Recurrence** (e.g., every day at 07:00), or
        - **When a file is created/modified** in **OneDrive/SharePoint** (if your CSV lands there).
    - **Action: Power BI ➜ Refresh a dataset**. Pick your **Workspace** and **Dataset**.

    - (Optional) Add **Delay** + **Get Refresh History** and send yourself a **Teams/Email** notification with the status.
3. If your data is on-prem (local/network drive), configure an **On-premises data gateway** and map the data source in the Service so refresh can reach it.

Extra: Change `OrderDate` correctly (DMY vs MDY)

Your sample shows:

```
OrderDate
1/5/2023
2/5/2023
...
```

To avoid day/month confusion, always use **Using Locale…** when converting:

- **DMY** (e.g., 1 May 2023): use **en-GB** or **ru-RU**.
- **MDY** (e.g., Jan 5, 2023): use **en-US**.

M example:

```
#"OrderDate as Date" = Table.TransformColumnTypes(#"Typed", {{"OrderDate",
type date}}, "en-GB")
```

Tip: Keep your project looking “hand-done”

- Name steps meaningfully (e.g., `#"Keep Product & Price"`, `#"Filtered Rows: Qty>1"`).
- Use **Remove Other Columns** (right-click) instead of “Select Columns” once — it reads naturally.
- Add short **Descriptions** to queries (Query Properties) explaining purpose.
- Keep transformations minimal and logical; avoid unnecessary steps.